4. Making a Template

Contents

`#let` 1
Programming elements 1
Everything show rule 2
Import from another file 2
Template recipe 2
Some little life hacks 2
Questions 3

`#let`

`#let lol = [_kek_]` – defines variable `#lol` that produces *kek*. A variable can contain: `"string", 666cm, 1488em, 69%, 420, false, center`, a function call or an expression, etc.

`#let my-func(a: 0, b: 0, divisor) = (a + b)/divisor` – defines a function with *named arguments* `a` and `b` that have default values 0, and a *positional argument* `divisor`. `#my-func(b: 4, 2)` produces 2.

Programming elements

- An *array* is a structure like `(a, b, c,...)`.
  - Can be empty: `()`, or have 1 element `(a, )` – a trailing comma is required.
  - Can be multiplied: `(1fr,) * 3` = `(1fr, 1fr, 1fr)`.
  - Can be unpacked with `..<array>`, e.g., `..([a], [b], [c])` = `[a], [b], [c]`.
- A *dictionary* is a structure like (key-1: [field-1], key-2: “field-2”, pidor: 1337,...). To read a field, use `#<dictionary>.<key>`.
- A *closure*, or *nameless function*, is a construction like `<input> => <output>`, e.g.: `x => x + 1` or `(a, b) => a + b`.
- A *method* is a function of the `<object>.<method>()` form (don’t confuse with a field access). Examples:

- `<array>.len()` – returns array's length.
- `<array>.map(<function>)` – returns a new array whose elements are the old elements processed by the function.
- `calc.min(<list of numbers>)` – returns the least number.
- `<function>.with(<list of arguments>)` – returns a new function with already filled arguments.

Everything show rule

`#show: <function>` – makes all that follows an argument of the function and returns the result. The function must be unary.

Import from another file

`#import "<file>": <name list>` – imports elements from the file. The imported element see not imported elements of the same file.

Template recipe

1. Create a separate file.
2. Make a template function whose last argument will take the main document. Use `{}` if necessary.
3. Import the function to the main file.
4. To fill up secondary arguments and make the function unary, use everything show rule with either:
   - a nameless function: `doc => conf(<arg1>, <arg2>, doc)`,
   - `.with()` method: `conf.with(<arg1>, <arg2>)`.
5. Type your document below the rule.

Some little life hacks

`#table(...)` function is similar to `#grid(...)`	`_#var_` won't work, use `_#var;_`
`$->$` creates →, and there're other shortcuts: `$-->$`, `$=>$`, `$<==>$`, etc.	`row-gutter: <length>` parameter controls gaps between rows.

Questions

1. Explain `#let`.
2. How to define a function? A variable?
3. What's an array? A dictionary?
4. Give an empty array. Give a 1-element array.
5. What're dictionary keys? Dictionary field? What does `#dictionary.key` produce?
6. What's an array multiplied by a number? What's `..array`?
7. What's a closure? Give a closure of 1 argument. Of several arguments.
8. What's a nameless function? Give a nameless function of several arguments.
9. What's a method?
10. Explain the following constructions: `.len()`, `.map()`, `.min()`, `.with()`.
11. Explain everything show rule? What kind of function does it require?
12. Explain `#import` function.
13. You imported function `#func` from `"template.typ"`. The file contains several functions and variables that are used in the body of `#func`, but haven't been imported. What will happen when you call `#func` in the main document? How to solve this problem?
14. Give a recipe of creating a table.
15. Explain how to provide a multi-argument function to everything show rule using: a) closure; b) `.with()`.
16. Explain `#table` function.
17. Why `_#var_` markupping won't work? How to resolve the issue?
18. Give shortcuts for: $\to$, $\longrightarrow$, $\Rightarrow$, $\Longleftrightarrow$. What do they require?
19. Explain `row-gutter: <...>` parameter.